Generate a full-stack **Library Management System** using the following technology stack:

Frontend: React + Bootstrap
Backend: Node.js + Express.js
Database: MongoDB with Mongoose ODM
Authentication: JWT (JSON Web Token)

The application should follow a **RESTful API architecture** and be structured as a scalable production-ready project.

Functional Requirements

User Roles

There must be two types of users:

1. Student
2. Administrator

Student Features

Students should be able to:

- Register and login
- View list of available books
- Borrow a book
- Return a book
- View borrowed books
- View due date for each borrowed book
- Check overdue fines if applicable

Administrator Features

Administrators should be able to:

- Login securely
- Add new books to the library
- Update book details
- Delete books
- View list of all registered students
- View all borrowed books
- Track which student borrowed which book
- Approve or reject student registrations if needed

Backend Requirements

The backend must include:

- RESTful API endpoints
- Proper HTTP status codes
- JSON responses
- Input validation
- Centralized error handling
- Middleware for authentication and authorization
- Role-based access control (RBAC)

Authentication should use **JWT tokens**.

Protected routes must verify the JWT token before allowing access.

MongoDB Database Design

Use MongoDB with Mongoose and create the following collections:

Users Collection

Fields:

- _id
- name
- email
- password (hashed)
- role (student or admin)
- createdAt

Books Collection

Fields:

- _id
- title
- author
- isbn
- category
- totalCopies
- availableCopies
- createdAt

BorrowRecords Collection

Fields:

- _id
- userId (reference to Users)
- bookId (reference to Books)
- borrowDate
- dueDate

- returnDate
- status (borrowed / returned / overdue)

Use **Mongoose relationships with ObjectId references**.

API Endpoints

Authentication:
POST /api/auth/register
POST /api/auth/login

Books:
GET /api/books
GET /api/books/:id
POST /api/books
PUT /api/books/:id
DELETE /api/books/:id

Borrowing:
POST /api/borrow
POST /api/return

Student Dashboard:
GET /api/student/borrowed-books
GET /api/student/due-books

Admin Dashboard:
GET /api/admin/borrowed-books
GET /api/admin/users

Project Structure

Backend folder structure should follow best practices:

```
backend/
├── controllers
├── models
├── routes
├── middleware
├── config
├── utils
└── server.js
```

Frontend folder structure:

```
frontend/
├── components
├── pages
├── services
```

```
├── context
└── App.js
```

Security Requirements

- Passwords must be hashed using bcrypt
- JWT authentication middleware must protect routes
- Input validation must be implemented
- Role-based authorization must restrict admin-only routes

Additional Features

Implement:

- Pagination for large book lists
- Search books by title or author
- Proper error messages
- API response structure standardization

Example API response format:

```
{
"success": true,
"message": "Book borrowed successfully",
"data": {...}
}
```

Testing Requirements

Generate a **Postman collection** to test all API endpoints.

Include:

- Login request
- Add book request
- Borrow book request
- Return book request

DevOps and Deployment

Include instructions for:

- Docker containerization
- GitHub repository structure
- Deployment to a cloud platform (Render / AWS / Railway)

Documentation

Generate a complete README including:

- Project overview
- Installation steps
- Environment variables
- API documentation
- Example requests and responses